

# Middleware NATS

Eduardo Silveira Alvim Santos

UFU

Universidade Federal de Uberlândia

Monte Carmelo - MG, Brasil

eduardo25@ufu.br

**Abstract**—This paper presents an overview and practical study of NATS, a lightweight and high-performance *Message-Oriented middleware* designed for distributed systems. The work introduces fundamental concepts of distributed architectures and *middleware*, explores the architecture, services, and operation of NATS, and demonstrates its practical application through installation and testing on a real environment. The goal is to provide students and professionals with both theoretical and hands-on understanding of how NATS simplifies communication, scalability, and resilience in modern distributed systems.

**Index Terms**—NATS, *middleware*, distributed systems, *Message-Oriented middleware*, *publish/subscribe*, *JetStream*

## I. INTRODUÇÃO

Os sistemas distribuídos surgiram como uma resposta à crescente demanda por escalabilidade, desempenho e disponibilidade em aplicações modernas. Nessas arquiteturas, múltiplos componentes independentes — muitas vezes localizados em diferentes máquinas e regiões — cooperam para oferecer serviços integrados e de alta performance. Esse modelo é amplamente adotado em arquiteturas baseadas em microsserviços, permitindo que empresas ampliem suas operações, aumentem a tolerância a falhas e garantam maior flexibilidade no desenvolvimento e na manutenção de suas aplicações.

Apesar de seus benefícios, a distribuição de componentes impõe desafios significativos, como a necessidade de coordenação, comunicação eficiente e gerenciamento de estados. Para superar essas dificuldades, torna-se essencial adotar soluções tecnológicas que assegurem o funcionamento coeso e confiável do sistema como um todo. Nesse cenário, o *middleware* se destaca por atuar como uma camada intermediária que conecta diferentes aplicações, serviços e sistemas operacionais.

Segundo a definição da Amazon Web Services (AWS), o *middleware* funciona como uma espécie de “cola” que possibilita a comunicação entre aplicações distribuídas, fornecendo serviços como mensageria, autenticação, roteamento e gerenciamento de APIs. Essa camada reduz a complexidade da integração entre sistemas heterogêneos, permitindo que os desenvolvedores concentrem seus esforços na lógica de negócio, sem se preocupar diretamente com protocolos e detalhes de interoperabilidade.

Entre as diversas soluções disponíveis, o NATS se destaca como uma plataforma de mensageria e *streaming* leve e de alto desempenho, voltada para arquiteturas distribuídas. Projetado para oferecer comunicação rápida e confiável entre microsserviços, dispositivos IoT e aplicações em nuvem, o

NATS suporta padrões como *publish/subscribe*, *request/reply* e *streaming* de dados. Com foco em escalabilidade horizontal e baixa latência, tornou-se uma alternativa atrativa para organizações que necessitam construir sistemas distribuídos robustos, eficientes e responsivos.

## II. FUNDAMENTAÇÃO TEÓRICA

Para compreender de forma aprofundada o funcionamento do NATS, é necessário primeiro entender os conceitos que sustentam sua arquitetura e justificam seu papel como *middleware* em sistemas distribuídos.

### A. *middleware Orientado a Mensagens*

O NATS é classificado como um *middleware* orientado a mensagens (*Message-Oriented middleware* – MOM). Esse tipo de solução atua como uma camada intermediária responsável por facilitar a comunicação entre aplicações e serviços distribuídos, evitando que os sistemas precisem estabelecer conexões diretas entre si.

No modelo MOM, a troca de informações ocorre por meio de mensagens enviadas a canais lógicos — conhecidos como *topics* ou *subjects* — aos quais diferentes aplicações podem se conectar para publicar ou consumir dados. Esse padrão promove o desacoplamento entre sistemas, já que produtores e consumidores de mensagens não precisam conhecer detalhes sobre a localização ou implementação uns dos outros, interagindo apenas por meio do *middleware*.

Entre os principais benefícios desse modelo estão a flexibilidade, que facilita a integração de sistemas heterogêneos; a escalabilidade, que suporta grandes volumes de mensagens; e a resiliência, uma vez que falhas em componentes individuais não comprometem toda a comunicação.

### B. *Sistemas Distribuídos e Comunicação Assíncrona*

O uso de *middlewares* está diretamente relacionado ao contexto de sistemas distribuídos, nos quais diversos componentes — frequentemente executados em diferentes máquinas, regiões ou provedores de nuvem — colaboram para entregar serviços completos.

Nesses cenários, a comunicação assíncrona é amplamente adotada, pois possibilita que as aplicações troquem informações sem a necessidade de respostas imediatas. Essa abordagem reduz o acoplamento e amplia o paralelismo, tornando os sistemas mais ágeis e escaláveis. Além disso,

favorece a implementação de arquiteturas orientadas a eventos (*Event-Driven Architectures*), nas quais serviços reagem dinamicamente a mensagens ou eventos recebidos.

### C. Conceitos de Publicação e Assinatura (*publish/subscribe*)

Um dos padrões mais comuns utilizados por *middlewares* de mensagens é o modelo *publish/subscribe* (Pub/Sub). Nesse paradigma:

- **Publicadores (*publishers*):** enviam mensagens para um canal ou *subject*, sem necessidade de conhecer os destinatários;
- **Assinantes (*subscribers*):** registram interesse em determinados canais, processando as mensagens assim que são recebidas.

Esse modelo reduz a complexidade da integração entre sistemas, pois elimina a necessidade de comunicação ponto a ponto, e permite que novos serviços sejam adicionados ao ecossistema sem impactar os já existentes.

### D. Qualidade de Serviço (QoS) em Mensageria

Outro conceito essencial em *middlewares* de mensagens é a Qualidade de Serviço (*Quality of Service* – QoS), que define as garantias relacionadas à entrega de mensagens. As mais comuns são:

- **At Most Once (no máximo uma vez):** a mensagem é enviada sem garantia de entrega; caso o receptor esteja indisponível, ela pode ser perdida.
- **At Least Once (pelo menos uma vez):** a entrega é garantida, porém a mensagem pode ser recebida mais de uma vez, exigindo que o consumidor trate duplicatas.
- **Exactly Once (exatamente uma vez):** assegura que cada mensagem seja entregue apenas uma vez, eliminando duplicações, embora com maior complexidade e custo operacional.

Essas garantias são determinantes para a forma como o *middleware* opera e impactam diretamente a maneira como aplicações que dependem dele são projetadas.

## III. OPERAÇÃO

O NATS é um *middleware* de mensagens desenvolvido para interligar aplicações e serviços distribuídos de maneira simples, confiável e com baixa latência. Sua arquitetura foi projetada para abstrair desafios típicos de sistemas distribuídos — como descoberta de serviços, balanceamento de carga, tolerância a falhas e escalabilidade —, permitindo que os desenvolvedores implementem soluções orientadas a eventos e comunicação assíncrona sem lidar diretamente com a complexidade da infraestrutura subjacente. A seguir, apresentam-se seus principais recursos, arquitetura e funcionamento em ambientes práticos.

### A. Recursos Fundamentais

O NATS oferece um conjunto abrangente de funcionalidades que simplificam a comunicação entre sistemas de forma escalável e desacoplada. Seus principais recursos incluem:

- **Publicação e Assinatura (*publish/subscribe*):** possibilita que aplicações publiquem mensagens em *subjects* (canais lógicos), que são recebidas por consumidores interessados em tempo real ou sob demanda. Esse modelo é amplamente utilizado em microsserviços e IoT, nos quais eventos — como leituras de sensores ou notificações de transações — precisam ser distribuídos a diversos serviços sem vínculos diretos.
- **Requisição e Resposta (*request/reply*):** permite comunicação ponto a ponto entre serviços. Um cliente envia uma solicitação a um *subject* e recebe resposta de qualquer instância disponível, recurso útil em arquiteturas orientadas a serviços (SOA) e para chamadas internas de APIs, eliminando a necessidade de balanceadores dedicados.
- **Armazenamento de Estado Compartilhado e Dados Temporais:** possibilita, juntamente ao *JetStream*, persistir eventos e manter estados distribuídos, viabilizando *replay* (reprocessamento de eventos) e armazenamento em estruturas como *key-value stores*.
- **Notificações e Processamento Assíncrono em Tempo Real:** garante que sistemas reativos sejam informados assim que novos eventos ocorrem, assegurando respostas rápidas e eficientes.
- **Resiliência e Reconexão Automática:** em caso de falhas, clientes podem se reconectar a outros servidores do *cluster* automaticamente, reduzindo a indisponibilidade.
- **Qualidade de Serviço Avançada (QoS):** com o *JetStream*, o NATS suporta garantias como *At Least Once* (entrega garantida, mesmo com duplicatas) e *Exactly Once* (eliminação de duplicações com confirmações explícitas).

Esses recursos tornam o NATS competitivo em relação a outras soluções, como RabbitMQ — focado em filas tradicionais — e Kafka — voltado ao processamento de grandes volumes de dados —, destacando-se pela leveza, simplicidade e desempenho em tempo real.

### B. Arquitetura do NATS

O funcionamento do NATS baseia-se em dois elementos principais: clientes (aplicações) e servidores (*nats-server*).

#### 1) Clientes NATS:

- São aplicações que utilizam bibliotecas oficiais em linguagens como Go, Java, Python e outras, para se conectar à infraestrutura do NATS.
- As bibliotecas oferecem APIs para publicação de mensagens, assinatura de *subjects*, execução de requisições e utilização do *JetStream*.
- Os clientes podem ser microsserviços, sistemas IoT, *pipelines* de dados, aplicações web ou qualquer software que exija comunicação em tempo real.

#### 2) Infraestrutura de Servidores (*nats-server*):

- Cada instância é leve (menos de 20 MB), podendo ser executada em dispositivos com recursos limitados ou em *superclusters* distribuídos em múltiplos provedores de nuvem.

- Os servidores se interconectam automaticamente, formando *clusters* e *superclusters* capazes de replicar mensagens, balancear conexões e tolerar falhas sem intervenção manual.

### C. Conexão e Autenticação

Para uma aplicação se conectar ao NATS, alguns elementos básicos são necessários:

- **URL NATS:** define o servidor ou *cluster* de destino (IP, porta e protocolo, como TCP, TLS ou WebSocket).
- **Autenticação:** pode ser realizada por usuário e senha, tokens, certificados TLS, JWT descentralizado ou chaves criptográficas NKey com desafio-resposta.

Após a conexão, a aplicação obtém um objeto que centraliza todas as operações, incluindo publicação, assinatura e acesso ao *JetStream*. As bibliotecas também implementam reconexão automática, garantindo a continuidade do serviço mesmo em casos de falhas nos servidores.

### D. Modelo de Mensagens

O modelo de mensagens do NATS combina simplicidade e flexibilidade:

- Cada mensagem possui um *subject* (canal lógico), um *payload* (dados binários ou serializados) e, opcionalmente, cabeçalhos e campo de resposta (*reply-to*).
- As mensagens podem ser processadas de forma assíncrona ou síncrona, dependendo da API da linguagem utilizada.
- Quando integradas ao *JetStream*, podem exigir confirmações (*acknowledgements*), permitindo *redelivery* em caso de falha e evitando perda de dados.

Esse design possibilita lidar com grandes volumes de tráfego com latência reduzida, mesmo em arquiteturas distribuídas globalmente.

### E. Publicação, Assinatura e Grupos de Fila

Com uma conexão ativa, uma aplicação pode:

- **Publicar (*publish*):** enviar mensagens para um *subject* sem necessidade de conhecer os consumidores.
- **Assinar (*subscribe*):** registrar interesse em mensagens de um ou mais *subjects*, com suporte a *wildcards* para capturar múltiplos canais.
- **Participar de Grupos de Fila (*Queue Groups*):** assinantes podem se agrupar logicamente, permitindo que o NATS distribua mensagens entre eles e implemente balanceamento de carga automático.

Esse modelo favorece o processamento paralelo de tarefas, *pipelines* de eventos e a distribuição massiva de dados em tempo real.

### F. JetStream

O *JetStream* amplia as funcionalidades do NATS, adicionando recursos de persistência e processamento distribuído:

- **Streams:** armazenam mensagens publicadas para consumo posterior ou em tempo real, sendo úteis para *replay* de eventos históricos e filas de trabalho distribuídas.

- **Consumidores duráveis e efêmeros** consumidores duráveis mantêm estado e podem ser reutilizados, enquanto os efêmeros são criados temporariamente, úteis para testes ou tarefas pontuais.
- **Key-Value Store:** possibilita armazenar pares chave-valor com histórico e monitoramento em tempo real.
- **Object Store:** permite o armazenamento de objetos de maior porte, superando limites de tamanho de mensagens.
- **Qualidade de Serviço Avançada:** suporta *At Least Once* e *Exactly Once*, tornando-o adequado para sistemas críticos.

Com esses recursos, o NATS atua como uma solução híbrida de mensageria, filas e armazenamento distribuído de eventos, competindo com plataformas como Kafka e AWS Kinesis, mas com maior leveza e simplicidade operacional.

### G. Operações de Requisição e Resposta

Além do modelo Pub/Sub, o NATS simplifica a implementação de chamadas de serviços distribuídos:

- Um cliente requisitante utiliza o método *request*, publicando uma mensagem com endereço de resposta (*reply-to*).
- Serviços respondentes assinam o *subject* correspondente, processam a solicitação e retornam uma resposta.
- Múltiplas instâncias de serviços podem participar como grupos de fila, balanceando a carga e aumentando a tolerância a falhas.

Esse padrão elimina a necessidade de balanceadores externos ou registros centralizados de serviços, já que o próprio NATS gerencia a distribuição dinamicamente.

## IV. TESTE

Esta seção apresenta um guia prático para instalação e configuração do NATS em um ambiente Ubuntu, utilizando o Docker, seguido de um teste prático que demonstra seu funcionamento em um cenário simples de comunicação entre serviços distribuídos. Essa abordagem permite não apenas preparar o ambiente de forma ágil, evitando configurações complexas, como também validar, na prática, os conceitos abordados sobre comunicação assíncrona e desacoplamento entre sistemas.

### A. Instalação do Docker

Antes de configurar o NATS, é necessário garantir que o Docker esteja instalado na máquina. O processo envolve os seguintes passos:

- 1) Atualizar os pacotes do sistema.
- 2) Instalar as dependências necessárias.
- 3) Adicionar a chave GPG oficial do Docker.
- 4) Adicionar o repositório do Docker.
- 5) Atualizar o índice de pacotes e instalar o Docker.

```
sudo apt update && sudo apt upgrade -y
```

Fig. 1. Comando de atualização de pacotes

```
sudo apt install apt-transport-https ca-certificates curl
software-properties-common -y
```

Fig. 2. Comando para instalar dependências necessárias

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo gpg --dearmor -o /usr/share/keyrings/docker.gpg
```

Fig. 3. Comando para adicionar a chave GPG

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/
usr/share/keyrings/docker.gpg] https://
download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /
dev/null
```

Fig. 4. Comando para adicionar o repositório do Docker

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io -y
```

Fig. 5. Comandos para atualizar índice de pacotes e instalar o Docker

A execução sequencial desses comandos assegura que o ambiente esteja preparado para rodar containers de forma estável e compatível.

## B. Instalação do NATS

Com o Docker instalado, o próximo passo é configurar o NATS. Para isso:

- 1) Baixe a imagem oficial do NATS por meio do Docker.
- 2) Execute o NATS Server em um *container*.
- 3) Após iniciado, acesse o painel administrativo em <http://localhost:8222/>.
  - Essa interface disponibiliza informações sobre conexões ativas, métricas e status geral do servidor, auxiliando no monitoramento e diagnóstico do ambiente.

```
docker pull nats:latest
```

Fig. 6. Comando para baixar a imagem oficial do NATS

```
docker run -d --name nats-server -p 4222:4222 -p 8222:8222 nats:latest
```

Fig. 7. Comando para executar o NATS Server

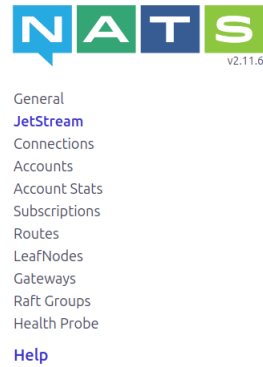


Fig. 8. Interface web do NATS

## C. Teste Prático

Para ilustrar o uso do NATS, será desenvolvido um cenário simples com dois serviços:

- 1) **Publicador:** envia mensagens, simulando um serviço que gera eventos.
- 2) **Assinante:** recebe as mensagens em tempo real, representando um serviço que consome eventos.

Esse experimento demonstra como o NATS facilita a comunicação assíncrona e o desacoplamento entre sistemas, permitindo que os serviços interajam sem conhecer detalhes técnicos uns dos outros.

Os passos para execução são:

- 1) Garantir que o NATS esteja rodando via Docker.
- 2) Criar dois scripts em Python (publicador e assinante, conforme apresentado no artigo).

```
1 import asyncio
2 import nats
3
4 async def main():
5     # Conecta ao servidor NATS
6     nc = await nats.connect("nats://localhost:4222")
7     print("Publicador conectado ao NATS.")
8
9     # Envia mensagens de teste a cada 2 segundos
10    for i in range(5):
11        message = f"Evento número {i+1}"
12        await nc.publish("eventos.teste", message.encode())
13        print(f"Mensagem publicada: {message}")
14        await asyncio.sleep(2)
15
16    # Fecha a conexão
17    await nc.drain()
18
19 if __name__ == "__main__":
20    asyncio.run(main())
```

Fig. 9. Código do publicador (Python)

```

1 import asyncio
2 import nats
3
4 async def message_handler(msg):
5     subject = msg.subject
6     data = msg.data.decode()
7     print(f'Mensagem recebida em '{subject}': {data}')
8
9 async def main():
10     # Conecta ao servidor NATS
11     nc = await nats.connect("nats://localhost:4222")
12     print("Assinante conectado ao NATS.")
13
14     # Inscreve-se no canal "eventos.teste"
15     await nc.subscribe("eventos.teste", cb=message_handler)
16
17     # Mantém o programa rodando para ouvir as mensagens
18     while True:
19         await asyncio.sleep(1)
20
21 if __name__ == "__main__":
22     asyncio.run(main())

```

Fig. 10. Código do assinante (Python)

- 3) Abrir dois terminais: executar o assinante em um e, em seguida, o publicador em outro.

```

Assinante conectado ao NATS.
Mensagem recebida em 'eventos.teste': Evento número 1
Mensagem recebida em 'eventos.teste': Evento número 2
Mensagem recebida em 'eventos.teste': Evento número 3
Mensagem recebida em 'eventos.teste': Evento número 4
Mensagem recebida em 'eventos.teste': Evento número 5
□

```

Fig. 11. Execução do assinante recebendo eventos

```

Publicador conectado ao NATS.
Mensagem publicada: Evento número 1
Mensagem publicada: Evento número 2
Mensagem publicada: Evento número 3
Mensagem publicada: Evento número 4
Mensagem publicada: Evento número 5

```

Fig. 12. Execução do publicador enviando eventos

Durante o teste, é possível observar:

- **Desacoplamento:** o publicador não precisa conhecer a quantidade ou a localização dos assinantes.
- **Escalabilidade:** múltiplos assinantes podem ser adicionados, com distribuição automática das mensagens ou balanceamento via *queue groups*.
- **Comunicação Assíncrona:** os eventos continuam sendo publicados independentemente do ritmo de processamento dos assinantes.
- **Extensibilidade:** o padrão pode ser expandido para cenários mais complexos, como integrações com microsserviços e dispositivos IoT.

Esse teste evidencia, de forma prática, como o NATS possibilita a criação de sistemas distribuídos escaláveis, reativos e resilientes.

## V. CONSIDERAÇÕES FINAIS

O estudo e a aplicação prática do NATS evidenciam a relevância de soluções de *middleware* orientadas a mensagens no desenvolvimento de sistemas distribuídos modernos. Sua arquitetura, caracterizada pela simplicidade de configuração, baixa latência e suporte a diferentes padrões de comunicação — como *publish/subscribe* e *request/reply* —, demonstra que o NATS é uma ferramenta eficaz para conectar microsserviços,

aplicações em nuvem e dispositivos IoT em cenários que exigem escalabilidade e desempenho.

A análise realizada permitiu compreender não apenas as funcionalidades do NATS, mas também como ele contribui para abstrair a complexidade da comunicação entre sistemas heterogêneos, facilitando a construção de arquiteturas resilientes e flexíveis. Recursos avançados, como o *JetStream*, que adiciona persistência de mensagens, garantias de qualidade de serviço e armazenamento distribuído, ampliam significativamente suas possibilidades de uso em contextos críticos, como processamento de eventos em larga escala e integração de sistemas corporativos.

O experimento prático desenvolvido, utilizando Python e Docker, demonstrou que o NATS pode ser implementado de maneira rápida e funcional, mesmo em ambientes simplificados. Essa experiência não apenas reforçou conceitos como comunicação assíncrona, desacoplamento de serviços e uso de padrões de integração, mas também mostrou como esses princípios se traduzem em soluções reais, aplicáveis a cenários corporativos e acadêmicos.

Conclui-se, portanto, que o estudo e a utilização do NATS não apenas ampliam o repertório técnico do desenvolvedor, mas também fornecem uma base sólida para a compreensão de tecnologias mais complexas no futuro. Ao permitir a criação de sistemas escaláveis, eficientes e adaptáveis, o NATS se consolida como uma solução de destaque para o desenvolvimento de aplicações distribuídas em um mercado cada vez mais orientado a eventos e dados.

## REFERENCES

- [1] Atlassian, “O que é um sistema distribuído?”, disponível em: <https://www.atlassian.com/br/microservices/microservices-architecture/distributed-architecture>.
- [2] Amazon Web Services, “O que é *middleware*?”, disponível em: <https://aws.amazon.com/pt/what-is/middleware/>.
- [3] NATS.io, “The official NATS documentation”, disponível em: <https://docs.nats.io/>.